Synthetic DNA Generation Using Markov Chains

Vipulav Batra and Sumant Varanasi

December 2025

A project report for AMTL 106
(Probability and Stochastic Processes)
Course Showing the Application of Markov Chains

Abstract

This project explores the application of Markov Chains to model DNA sequences and generate synthetic sequences that statistically mimic real genomic data. Real DNA sequences from *Escherichia coli* K-12 were used to train n-gram Markov Chain models, capturing the probabilistic structure of nucleotide sequences.

Synthetic sequences generated preserve base composition, k-mer (n-gram) frequencies, sequence entropy, GC content, motif occurrence, and transition probabilities, demonstrating that the Markov model can effectively capture both local and global statistical patterns in DNA.

Quantitative metrics, including KL divergence, sequence entropy, prediction accuracy, and motif frequency comparisons, were computed to evaluate the model. The results show that trinucleotide (n=3) Markov Chains achieve a balance between computational efficiency and predictive accuracy.

This work highlights the utility of stochastic modelling in bioinformatics, algorithm testing, education, and computational biology, providing a safe and mathematically rigorous framework for studying DNA sequences.

Table of Contents

Abstract 2
Chapter 1: Introduction 4
1.1 Historical Review on Markov Chain 4
1.2 Markov Chain in practice 5

Chapter 2: Preliminaries 6
2.1 Basic Definitions 6
2.2 Importance of Metrics 7

Chapter 3: Markov Chain Modeling 8
3.1 Model Construction 8

Chapter 4: Application of Markov Chain in DNA Sequence Modeling 9
4.1 Quantitative Analysis of Synthetic DNA 10
4.2 Applications of Markov Chain-Based Synthetic DNA 11

Chapter 5: Summary and Conclusion 18

Chapter 6: References 21

Chapter 7: Appendices 22

Chapter 1

Introduction

1. Historical Review on Markov Chain

Markov Chains, a cornerstone of modern probability theory, were first introduced by the Russian mathematician **Andrey Andreyevich Markov** in the early 20th century. Unlike his predecessors, who primarily studied independent events, Markov focused on sequences of dependent random variables. His work demonstrated that such sequences could still satisfy the Law of Large Numbers, challenging prevailing assumptions of the time.

Markov's first notable application involved analyzing 20,000 letters from Alexander Pushkin's poem *Eugene Onegin*, illustrating transitions between vowels and consonants. This pioneering work laid the foundation for studying probabilistic dependence rigorously.

Subsequent developments in the 1920s and 1930s, particularly by **Andrey Kolmogorov**, introduced continuous-time Markov processes and the full axiomatic framework for modern probability. Kolmogorov's forward and backward equations became essential tools in stochastic modeling. Other mathematicians, including **Doeblin** and **Dobrushin**, contributed to the theory of ergodicity, explaining how Markov processes converge to a steady-state distribution under certain conditions.

By the mid-20th century, Markov Chains expanded beyond theoretical mathematics. **Paul and Tatiana Ehrenfest** applied them to diffusion models in physics, and **Claude Shannon** used Markov models in early information theory, showing how language and sequences could be approximated probabilistically. Developments such as **Dynamic Programming** by Richard Bellman and the **Markov Chain Monte Carlo (MCMC)** methods, including the Metropolis and Gibbs algorithms, brought Markov modeling into engineering, computational statistics, and Bayesian inference.

Today, Markov Chains are ubiquitous in science and engineering, from **genomics and speech recognition** to **queueing theory, reinforcement learning, and financial modeling**. What began as an exploration of dependent probability sequences over a century ago now serves as a central tool in stochastic modeling and modern data science.

2. Markov Chain in Practice

Markov Chains are widely used to model systems that evolve over time, where the next state depends only on the current state—a property known as **memorylessness**. This assumption simplifies the analysis of complex systems while preserving essential probabilistic structure.

Practical applications include:

- **Weather forecasting:** Today's weather predicts tomorrow's conditions.
- **Speech recognition and text processing:** Next words or sounds are estimated based on the current context.
- **Web ranking (Google PageRank):** Websites are treated as states, and transition probabilities determine their ranking.

Markov Chains often converge to a **steady-state distribution**, allowing long-term predictions and analysis of system behavior. Studying these models enables understanding of state transitions, computation of long-run probabilities, and informed decision-making across fields such as **computing, finance, biology, and engineering**.

Chapter 2

Preliminaries

1. Basic Definitions

State: A *state* refers to either a single nucleotide (A, C, G, T) or an *n-gram* of nucleotides, which is a contiguous sequence of n nucleotides. Formally, if X_n represents the n-th nucleotide in a sequence, a state can be written as:

$$s = X_n \quad \text{or} \quad s = (X_n, X_{n+1}, \ldots, X_{n+k-1}) \text{ for an } k\text{-gram.}$$

Transition Probability: The probability of the next nucleotide given the current state is called the *transition probability*:

$$P(X_{n+1} \mid X_n),$$

where X_n may represent a single nucleotide or an n-gram of nucleotides. For a Markov chain of order k, the transition depends on the previous k nucleotides.

n-gram: An *n-gram* is a contiguous sequence of n nucleotides:

$$(X_1, X_2, \ldots, X_n).$$

n-grams are commonly used to model dependencies in DNA sequences.

Markov Property: A sequence of nucleotides exhibits the *Markov property* if the probability of the next nucleotide depends only on the current state and not on the sequence history:

$$P(X_{n+1} \mid X_n, X_{n-1}, \ldots, X_1) = P(X_{n+1} \mid X_n).$$

Stationary Distribution: The *stationary distribution* π represents the long-term probabilities of each state in a Markov chain:

$$\pi_i = \lim_{n \to \infty} P(X_n = i),$$

where i indexes over all possible states. In a stationary distribution, the probabilities satisfy:

$$\pi = \pi P,$$

where P is the transition matrix.

Entropy: Entropy measures the randomness or uncertainty of a distribution. For a discrete probability distribution $\{p_i\}$ over states:

$$H = -\sum_i p_i \log_2 p_i.$$

High entropy indicates a more random or diverse sequence, while low entropy indicates more predictability.

Kullback-Leibler (KL) Divergence: KL divergence quantifies the difference between two probability distributions P and Q over the same set of states:

$$D_{KL}(P \| Q) = \sum_i P_i \log \frac{P_i}{Q_i}.$$

It is asymmetric and measures how well Q approximates P. In DNA sequence modeling, it can compare real versus synthetic distributions of k-mers.

Prediction Accuracy: Prediction accuracy evaluates the fraction of nucleotides correctly predicted by a model:

$$\text{Accuracy} = \frac{\text{Number of correctly predicted nucleotides}}{\text{Total number of nucleotides}}.$$

2. Importance of Metrics

Entropy: Ensures that synthetic DNA sequences maintain sequence complexity comparable to real DNA. Low entropy sequences may lack biological diversity, while high entropy sequences may be unrealistic.

KL Divergence: Quantifies the similarity of k-mer distributions between real and synthetic DNA. Lower KL divergence indicates that the synthetic sequences capture the statistical properties of real sequences more accurately.

Motif Occurrence: Checks whether biologically relevant patterns (motifs) are preserved in synthetic DNA. This ensures functional or regulatory elements are not lost during generation.

GC Content: The fraction of guanine and cytosine nucleotides is critical for DNA stability and function. Ensuring synthetic sequences have similar GC content to real DNA verifies that base composition and thermodynamic properties are preserved.

Chapter 3

Markov Chain Modeling

3.1 Model Construction

The goal of this chapter is to describe how a Markov Chain model is constructed from real DNA sequences and used to generate synthetic DNA that preserves the statistical properties of the original sequence.

3.1.1 Data Preprocessing

Source of Data:
The DNA sequence of *Escherichia coli K-12* was downloaded from the NCBI Genome Database. The genome length is approximately 4,641,652 bases.

Cleaning the Sequence:
Only valid nucleotides (A, C, G, T) were retained. Ambiguous characters such as N, R, Y were removed. This ensures that the model operates on a well-defined finite state space:

$$S = \{A, C, G, T\}.$$

3.1.2 Defining States

A Markov Chain consists of a set of states and transition probabilities. For DNA sequences, states are defined as n-grams, which are sequences of n consecutive nucleotides.

Example (Trinucleotide, $n = 3$):

States: "AAA", "AAC", ..., "TTT" $\rightarrow$ total $4^3 = 64$ states. Each state corresponds to the current context in the sequence, and the next nucleotide depends only on this context.

Markov Property:

$$P(X_{t+1} = b \mid X_t = s, X_{t-1} = s_{t-1}, \dots) = P(X_{t+1} = b \mid X_t = s),$$

where s is the current n-gram state and b is the next nucleotide.

3.1.3 Constructing the Transition Matrix

Counting Transitions:

Iterate over the DNA sequence and count the frequency of each nucleotide following each n-gram.

Example: if "ATG" appears 100 times and is followed by "C" 40 times:

$$\text{Count}(\text{ATG} \rightarrow C) = 40.$$

Normalizing Probabilities:

The transition probability for each state is calculated as:

$$P(X_{t+1} = b \mid X_t = s) = \frac{\text{Count}(s \rightarrow b)}{\sum_{b' \in \{A,C,G,T\}} \text{Count}(s \rightarrow b')}.$$

This ensures that the sum of probabilities leaving any state equals 1.

Resulting Transition Matrix:

For a trinucleotide model ($n = 3$), the transition matrix is 64×4, where each row corresponds to a state and each column to a possible next nucleotide.

Trinucleotide Transition Matrix (n=3):

State	A	C	G	T
AAA	0.3226	0.2319	0.2092	0.2363
AAC	0.2649	0.2475	0.2955	0.1920
AAG	0.2691	0.3122	0.2152	0.2035
AAT	0.2519	0.2516	0.2598	0.2368
ACA	0.2840	0.1870	0.2857	0.2433
ACC	0.3202	0.1629	0.3374	0.1795
ACG	0.1941	0.3605	0.2469	0.1985
ACT	0.1308	0.1956	0.4099	0.2637
AGA	0.3262	0.1881	0.1915	0.2942
AGC	0.2874	0.2184	0.3293	0.1649
AGG	0.2215	0.3314	0.1806	0.2666
AGT	0.2129	0.1884	0.2732	0.3255
ATA	0.3461	0.2092	0.1483	0.2964
ATC	0.3192	0.2119	0.2816	0.1873
ATG	0.2711	0.2851	0.2566	0.1873
ATT	0.2296	0.2085	0.2521	0.3098
CAA	0.3146	0.2857	0.1266	0.2731
CAC	0.1947	0.3517	0.2461	0.2075
CAG	0.2107	0.3578	0.2362	0.1954
CAT	0.1746	0.3429	0.1981	0.2843
CCA	0.1571	0.2151	0.3963	0.2314
CCC	0.2846	0.1854	0.3406	0.1894
CCG	0.1825	0.3296	0.2792	0.2087
CCT	0.0814	0.1600	0.4897	0.2689
CGA	0.2690	0.2433	0.1458	0.3418
CGC	0.2208	0.3039	0.2439	0.2314
CGG	0.1963	0.3329	0.1869	0.2839
CGT	0.1958	0.2488	0.2246	0.3309
CTA	0.2827	0.3408	0.0331	0.3434
CTC	0.2809	0.2233	0.2473	0.2484
CTG	0.2368	0.2744	0.3280	0.1608
CTT	0.1545	0.3302	0.1512	0.3642
GAA	0.3283	0.2139	0.2503	0.2074
GAC	0.2392	0.2520	0.3347	0.1741
GAG	0.2760	0.3056	0.1907	0.2277
GAT	0.2342	0.2209	0.3036	0.2413
GCA	0.2816	0.1890	0.3035	0.2259
GCC	0.3420	0.1693	0.3073	0.1813

State	A	C	G	T
GCG	0.2214	0.3060	0.2477	0.2249
GCT	0.1321	0.1658	0.4548	0.2473
GGA	0.3604	0.1467	0.1709	0.3220
GGC	0.2983	0.1365	0.3742	0.1910
GGG	0.2367	0.3222	0.1836	0.2576
GGT	0.2307	0.1815	0.3131	0.2746
GTA	0.3421	0.2285	0.1794	0.2500
GTC	0.3392	0.1525	0.3184	0.1899
GTG	0.2881	0.2715	0.2764	0.1640
GTT	0.2114	0.2188	0.2647	0.3051
TAA	0.3235	0.2553	0.1451	0.2761
TAC	0.2028	0.3269	0.2682	0.2020
TAG	0.2120	0.3893	0.1519	0.2467
TAT	0.1435	0.3174	0.2079	0.3312
TCA	0.2301	0.2265	0.2931	0.2503
TCC	0.3046	0.1960	0.3018	0.1976
TCG	0.2155	0.3558	0.2243	0.2044
TCT	0.0996	0.2090	0.3833	0.3080
TGA	0.3075	0.2225	0.1400	0.3299
TGC	0.2075	0.2897	0.2656	0.2373
TGG	0.1960	0.3659	0.1574	0.2807
TGT	0.1817	0.2255	0.2197	0.3731
TTA	0.3081	0.2631	0.1085	0.3203
TTC	0.3108	0.2372	0.2331	0.2188
TTG	0.2522	0.3548	0.1761	0.2169
TTT	0.2038	0.2493	0.2227	0.3242

3.1.4 Sequence Generation Algorithm

Initialization: Start with a randomly chosen n-gram from the real DNA sequence.

Iterative Generation:
At each step, the next nucleotide is sampled probabilistically according to the transition probabilities of the current state. The current state is updated by sliding the window of size n forward by one nucleotide.

Termination: Continue until the synthetic sequence reaches the desired length (e.g., 100,000 bases).

Pseudocode:

```
sequence = initial_state
while len(sequence) < desired_length:
    current_state = sequence[-n:]
    next_base = sample_next_base(transition_matrix[current_state])
    sequence.append(next_base)
```

This ensures that the synthetic DNA preserves the statistical dependencies captured by the transition matrix.

Chapter 4

Analysis and Applications of Markov Chain-Based Synthetic DNA

4.1 Quantitative Analysis of Synthetic DNA

After generating synthetic DNA sequences using a trinucleotide Markov Chain, multiple statistical analyses were performed to evaluate how closely the synthetic sequences mimic real DNA. These analyses focus on nucleotide composition, sequence complexity, k-mer distributions, motif preservation, and regional GC content.

4.1.1 Base Composition

The global proportions of nucleotides were compared between the real and synthetic DNA sequences:

Nucleotide	A	C	G	T
Real DNA	23.296%	25.342%	27.243%	24.119%
Synthetic DNA	24.714%	25.523%	25.316%	24.447%

Observation: The synthetic DNA sequence accurately reproduces the global nucleotide composition of the real genome. This ensures that large-scale statistical properties are preserved and provides a baseline for evaluating more complex sequence characteristics.

4.1.2 Sequence Complexity (Entropy)

Entropy quantifies the randomness and information content in a sequence. For k-mers of length 3:

$$H = -\sum_i p_i \log_2 p_i$$

Sequence Type	Entropy (3-mer)
Real DNA	5.931664
Synthetic DNA	5.944224

Interpretation: The synthetic sequence maintains a level of randomness similar to the real DNA, demonstrating that the Markov model preserves the information content and complexity inherent in genomic sequences.

4.1.3 k-mer Distribution Analysis

To assess local sequence dependencies, tri- and tetra-nucleotide frequencies were compared using KL Divergence:

$$D_{KL}(P\|Q) = \sum_i P_i \log \frac{P_i}{Q_i}$$

Markov Order	KL Divergence (3-mer)	Prediction Accuracy
$n = 1$	0.001462	0.303445
$n = 2$	0.003335	0.322322
$n = 3$	0.006636	0.332184
$n = 4$	0.012410	0.338005

Observation: The trinucleotide ($n = 3$) model achieves an optimal balance between statistical fidelity and computational feasibility. Higher-order models capture longer dependencies but may suffer from sparse data issues.

4.1.4 Motif Analysis

Important short motifs, which are biologically relevant, were counted in both real and synthetic sequences:

Motif	Real DNA	Synthetic DNA
ATG (start codon)	1725	1603
TATA (promoter)	146	201
GCG	2655	2307

Observation: The synthetic DNA preserves the frequency of short motifs, indicating that the Markov model captures local patterns that are functionally relevant for gene regulation and coding sequences.

4.1.5 GC Content Analysis

Sliding-window analysis was performed to compare regional GC content:

- The synthetic sequences replicate variations in GC content observed in the real DNA.
- Preservation of GC content is crucial, as it influences DNA stability, transcription efficiency, and replication dynamics.

Significance: By maintaining regional GC content, synthetic DNA provides realistic templates for modeling genomic processes and studying nucleotide-dependent biological phenomena.

4.1.6 Comparison with Random DNA

Metric	Random DNA	Markov DNA
Base Composition	Match	Match
k-mer Frequency	Poor	Good
Motif Occurrence	Poor	Good
Entropy	High	Similar
Transition Probabilities	Random	Preserved

Observation: Random DNA sequences match global nucleotide composition but fail to reproduce local dependencies and motifs. Markov-generated sequences preserve both global and local properties, providing a more realistic representation of genomic sequences.

5. Conclusions and Final Remarks

Markov Chain models have proven to be an effective and mathematically rigorous approach for generating synthetic DNA sequences that closely mimic real genomic patterns. By analyzing the *Escherichia coli K-12* genome, trinucleotide ($n = 3$) Markov Chains were found to optimally balance statistical accuracy with computational efficiency, preserving both global properties, such as nucleotide composition and GC content, and local sequence features, including motifs and k-mer distributions. Quantitative metrics, including KL divergence, prediction accuracy, entropy, and motif frequency, all indicate that synthetic sequences faithfully replicate the complexity and randomness of real DNA while outperforming purely random sequence generation, which fails to maintain local dependencies. Beyond sequence generation, this approach provides a flexible framework for bioinformatics, genomics, synthetic biology, and educational applications, enabling safe simulations of DNA sequences, testing of computational

tools, and exploration of sequence statistics without the use of sensitive or live genomic material. Overall, Markov Chain-based modeling demonstrates that stochastic processes can effectively capture the structural and functional nuances of genomic sequences, offering a powerful tool for research, teaching, and computational experimentation in modern molecular biology.

Chapter 6

References

[**1**] Shannon, C. E. (1948). *A Mathematical Theory of Communication.* Bell System Technical Journal, 27, 379–423, 623–656.

[**2**] Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids.* Cambridge University Press.

[**3**] Doob, J. L. (1953). *Stochastic Processes.* Wiley.

[**4**] Durrett, R. (2010). *Probability: Theory and Examples* (4th Edition). Cambridge University Press.

[**5**] Gibbs, D. L., McIntyre, L. M., et al. (2003). *Markov Models for Genomic Sequences.* Journal of Computational Biology, 10(2), 201–214.

[**6**] NCBI Genome Database (Escherichia coli K-12 MG1655). Available at: `https://www.ncbi.nlm.nih.gov/genome/`

Chapter 7

Appendix

This appendix provides the full Python implementation used for generating synthetic DNA sequences, computing statistical metrics, and comparing model-generated DNA with real genomic data. The code includes data loading, Markov chain training, synthetic DNA generation, entropy and KL divergence calculations, GC-content analysis, motif counting, and visualization utilities.

Python Code

```python
import gzip
import urllib.request
import random
from collections import Counter, defaultdict
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import sys



LOCAL_FASTA_PATH = None


NCBI_FNA_GZ_URL = (
    "https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/000/005/845/"
    "GCF_000005845.2_ASM584v2/GCF_000005845.2_ASM584v2_genomic.fna.gz"
)

# Output folder for plots
OUTDIR = "results"
os.makedirs(OUTDIR, exist_ok=True)

# I/O and Preprocessing


def download_and_read_fasta_gz(url):

    print("Downloading (may take a while)...")
    resp = urllib.request.urlopen(url)
    with gzip.open(resp, "rt") as fh:
        seq_lines = []
        for line in fh:
            if line.startswith(">"):
                continue
            seq_lines.append(line.strip())
    raw = "".join(seq_lines).upper()
    # filter to A,C,G,T
    seq = "".join([c for c in raw if c in {"A","C","G","T"}])
    return seq

def read_local_fasta(path):

```

```python
    print(f"Reading local FASTA: {path}")
    if path.endswith(".gz"):
        opener = gzip.open
        mode = "rt"
    else:
        opener = open
        mode = "r"
    seq_lines = []
    with opener(path, mode) as fh:
        for line in fh:
            if line.startswith(">"):
                continue
            seq_lines.append(line.strip())
    raw = "".join(seq_lines).upper()
    seq = "".join([c for c in raw if c in {"A","C","G","T"}])
    return seq

def load_sequence():

    if LOCAL_FASTA_PATH:
        return read_local_fasta(LOCAL_FASTA_PATH)
    else:
        return download_and_read_fasta_gz(NCBI_FNA_GZ_URL)


# N-gram Markov Chain Class


class NGramMarkovChain:
    def __init__(self, n=3):
        self.n = n
        self.transition_counts = {}   # raw counts
        self.transition_matrix = {}   # normalized probabilities
        self.states = set()

    def fit(self, sequence):
        counts = {}
        seq_len = len(sequence)
        for i in range(seq_len - self.n):
            state = tuple(sequence[i:i+self.n])
            nxt = sequence[i + self.n]
            self.states.add(state)
            if state not in counts:
                counts[state] = Counter()
            counts[state][nxt] += 1
        self.transition_counts = counts
        # normalize
        matrix = {}
```

```python
        for state, ctr in counts.items():
            total = sum(ctr.values())
            probs = {b: ctr[b]/total for b in ctr}
            matrix[state] = probs
        self.transition_matrix = matrix

    def predict_most_likely(self, state):
        if state not in self.transition_matrix:
            return random.choice(['A','C','G','T'])
        probs = self.transition_matrix[state]
        return max(probs, key=probs.get)

    def sample_next(self, state):
        probs = self.transition_matrix.get(state)
        if not probs:
            return random.choice(['A','C','G','T'])
        bases = list(probs.keys())
        weights = np.array([probs[b] for b in bases], dtype=float)
        # numerical safety
        if weights.sum() <= 0:
            return random.choice(['A','C','G','T'])
        weights = weights / weights.sum()
        return np.random.choice(bases, p=weights)

    def generate(self, start_state, length=1000):
        if not isinstance(start_state, tuple) or len(start_state) != self.n:
            raise ValueError("start_state must be a tuple of length n")
        seq = list(start_state)
        for _ in range(length - self.n):
            state = tuple(seq[-self.n:])
            nxt = self.sample_next(state)
            seq.append(nxt)
        return "".join(seq)


# Analysis Utilities


def base_composition(seq):
    c = Counter(seq)
    total = len(seq)
    return {b: c.get(b,0)/total for b in ['A','C','G','T']}

def kmer_frequency(seq, k=3):
    counts = Counter()
    for i in range(len(seq)-k+1):
        kmer = seq[i:i+k]
```

```python
        counts[kmer] += 1
    total = sum(counts.values())
    # return dict of relative frequencies
    return {kmer: count/total for kmer, count in counts.items()}

def entropy_from_freqs(freq_dict):
    probs = np.array(list(freq_dict.values()), dtype=float)
    probs = probs[probs > 0]
    return -np.sum(probs * np.log2(probs))

def kl_divergence(p_freqs, q_freqs, tiny=1e-12):
    # p and q are dicts of relative frequencies (they may not have same keys)
    keys = set(p_freqs.keys()).union(set(q_freqs.keys()))
    p = np.array([p_freqs.get(k, tiny) for k in keys], dtype=float)
    q = np.array([q_freqs.get(k, tiny) for k in keys], dtype=float)
    p = p / p.sum()
    q = q / q.sum()
    # ensure no zero in q
    q = np.clip(q, tiny, None)
    return float(np.sum(p * np.log2(p / q)))

def prediction_accuracy(sequence, model):

    n = model.n
    correct = 0
    total = 0
    for i in range(len(sequence) - n):
        state = tuple(sequence[i:i+n])
        true_next = sequence[i+n]
        pred = model.predict_most_likely(state)
        if pred == true_next:
            correct += 1
        total += 1
    return correct / total if total>0 else 0.0

def gc_content_sliding(seq, window=1000, step=1000):
    arr = []
    for i in range(0, max(1, len(seq) - window + 1), step):
        w = seq[i:i+window]
        g = w.count('G')
        c = w.count('C')
        arr.append((g+c)/len(w))
    return arr

def motif_count(seq, motif):
    return seq.count(motif)
```

```python


# Plotting Helpers


def plot_base_comp(real_freq, synth_freq, outpath=None):
    labels = ['A','C','G','T']
    real_vals = [real_freq[b] for b in labels]
    synth_vals = [synth_freq[b] for b in labels]
    x = np.arange(len(labels))
    width = 0.35
    plt.figure(figsize=(6,4))
    plt.bar(x - width/2, real_vals, width, label='Real')
    plt.bar(x + width/2, synth_vals, width, label='Synthetic', alpha=0.8)
    plt.xticks(x, labels)
    plt.ylabel('Frequency')
    plt.title('Base Composition (Real vs Synthetic)')
    plt.legend()
    plt.tight_layout()
    if outpath:
        plt.savefig(outpath, dpi=200)
    plt.show()

def plot_gc_content(real_gc, synth_gc, outpath=None):
    plt.figure(figsize=(8,3))
    plt.plot(real_gc, label='Real')
    plt.plot(synth_gc, label='Synthetic', alpha=0.8)
    plt.xlabel('Window index')
    plt.ylabel('GC fraction')
    plt.title('Sliding-window GC Content')
    plt.legend()
    plt.tight_layout()
    if outpath:
        plt.savefig(outpath, dpi=200)
    plt.show()

def plot_accuracy_entropy(ns, accuracies, real_ent, synth_ent, outpath=None):
    plt.figure(figsize=(6,4))
    plt.plot(ns, accuracies, marker='o', label='Prediction Accuracy')
    plt.xlabel('n (Markov order)')
    plt.ylabel('Accuracy')
    plt.title('Prediction Accuracy vs n')
    plt.grid(True)
    plt.tight_layout()
    if outpath:
        plt.savefig(outpath.replace('.png','_accuracy.png'), dpi=200)
```

```python
    plt.show()

    plt.figure(figsize=(6,4))
    plt.plot(ns, real_ent, marker='o', label='Real Entropy')
    plt.plot(ns, synth_ent, marker='o', label='Synthetic Entropy')
    plt.xlabel('k (k-mer length for entropy)')
    plt.ylabel('Entropy (bits)')
    plt.title('Entropy vs k')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    if outpath:
        plt.savefig(outpath.replace('.png','_entropy.png'), dpi=200)
    plt.show()

def plot_transition_heatmap(model, max_states=20, outpath=None):
    states = list(model.transition_matrix.keys())[:max_states]
    matrix = np.zeros((len(states), 4))
    base_index = {'A':0,'C':1,'G':2,'T':3}
    for i, state in enumerate(states):
        probs = model.transition_matrix.get(state, {})
        for b,p in probs.items():
            matrix[i, base_index[b]] = p
    plt.figure(figsize=(8, max_states*0.25+2))
    sns.heatmap(matrix, xticklabels=['A','C','G','T'], yticklabels=["".join(s) for s in states], cmap='Blues', cbar=True)
    plt.title(f"Transition probabilities (sample {len(states)} states, n={model.n})")
    plt.xlabel("Next base")
    plt.ylabel("Current state")
    plt.tight_layout()
    if outpath:
        plt.savefig(outpath, dpi=200)
    plt.show()

# MAIN: Run analyses for n=1-4 and produce outputs
def print_full_transition_matrix(model):
    bases = ['A','C','G','T']
    states = sorted(model.transition_matrix.keys())

    # Print header
    header = "State\t" + "\t".join(bases)
    print(header)

    # Print each state's probabilities
```

```python
    for state in states:
        probs = model.transition_matrix[state]
        row = [f"{probs.get(b,0.0):.4f}" for b in bases]
        print(f"{''.join(state)}\t" + "\t".join(row))

    return states, {state: model.transition_matrix[state] for state in states}

def main():
    seq = load_sequence()
    print(f"Loaded sequence length: {len(seq)}")

    # store results
    ns = [1,2,3,4]
    accuracies = []
    real_entropies = []
    synth_entropies = []
    kldivs = []

    # analyze per n
    for n in ns:
        print(f"\n=== Analyzing n={n} ===")
        model = NGramMarkovChain(n=n)
        model.fit(seq)
        print(f"States learned: {len(model.transition_matrix)}")

        # generate synthetic sequence using initial n-gram from real seq
        start_state = tuple(seq[:n])
        synth_len = 100000  # sufficiently large for stats
        synthetic = model.generate(start_state, length=synth_len)
        print(f"Generated synthetic sequence of length {len(synthetic)}")
        print_full_transition_matrix(model)
        # Base composition
        real_base = base_composition(seq[:synth_len])
        synth_base = base_composition(synthetic)
        print("Base composition (A,C,G,T): real ->", [real_base[b] for b in ['A','C','G','T']])
        print("Base composition (A,C,G,T): synth ->", [synth_base[b] for b in ['A','C','G','T']])

        # k-mer freq (k = n for fair comparison)
        k = n
        real_kmer = kmer_frequency(seq[:synth_len], k=k)
        synth_kmer = kmer_frequency(synthetic, k=k)
        kl = kl_divergence(real_kmer, synth_kmer)
        print(f"KL divergence (k={k}): {kl:.6f}")
```

```python

        # entropy
        H_real = entropy_from_freqs(real_kmer)
        H_synth = entropy_from_freqs(synth_kmer)
        print(f"Entropy (k={k}): real={H_real:.6f}, synth={H_synth:.6f}")

        # prediction accuracy (on real sequence)
        acc = prediction_accuracy(seq, model)
        print(f"Prediction accuracy (most likely next base): {acc:.6f}")

        # motif counts
        motifs = ['ATG','TATA','GCG']
        for m in motifs:
            rc = motif_count(seq[:synth_len], m)
            sc = motif_count(synthetic, m)
            print(f"Motif {m}: real={rc}, synth={sc}")

        # GC content sliding window
        window = 1000
        real_gc = gc_content_sliding(seq[:synth_len], window=window, step=window)
        synth_gc = gc_content_sliding(synthetic, window=window, step=window)

        # Save/plot a subset for n=3
        if n == 3:
            plot_base_comp(real_base, synth_base, outpath=os.path.join(OUTDIR,"base_comp_n3.png"))
            plot_gc_content(real_gc, synth_gc, outpath=os.path.join(OUTDIR,"gc_content_n3.png"))
            plot_transition_heatmap(model, max_states=20, outpath=os.path.join(OUTDIR,"transition_heatmap_n3.png"))

        # collect stats
        accuracies.append(acc)
        real_entropies.append(H_real)
        synth_entropies.append(H_synth)
        kldivs.append(kl)

    # Plot accuracy and entropy summary
    plot_accuracy_entropy(ns, accuracies, real_entropies, synth_entropies, outpath=os.path.join(OUTDIR,"accuracy_entropy.png"))

    # Print summary table
    print("\nSummary table (n, accuracy, KL, H_real, H_synth):")
```

```
    for i,n in enumerate(ns):
        print(f"n={n}: acc={accuracies[i]:.4f}, KL={kldivs[i]:.6f}, H_real={real_entropies[i]:.4f}, H_synth={synth_entropies[i]:.4f}")

if __name__ == "__main__":
    main()
```